Optimization of dense matrix multiplication and use of sparse matrices (CSR) in Java

Big Data - Assignment 2 - GCID, ULPGC, 2026-2027

Author: Pablo Mellado Martín

GitHub: [Assignment02_MelladoMartin_MatrixMultiplication]

Abstract

Two issues related to matrix multiplication in Java are analyzed: the extent to which performance improves when applying two optimization techniques—changing the loop order to i-k-j and using block-based multiplication—compared to the standard i-j-k method used in the initial task; and the point at which storing matrices using the CSR (Compressed Sparse Row) format ceases to be advantageous. For n = 1600, the i-k-j version proved to be 16 times faster than the standard approach, while the block-based version was 20 times faster. In the case of sparse matrices with uniformly distributed non-zero elements, multiplication using the CSR format was up to 1000 times faster than multiplication for dense matrices with a density level of 0.1%; however, it ceased to be superior at density levels between 25% and 50% (for n = 1000) and between 50% and 75% (for n = 2000). The results obtained are applicable only to these specific programming approaches, these input data, and this particular computing hardware.

1. Introduction and objetive

In Assignment 1, we discovered that the i-j-k method performs much more slowly when matrix B does not fit into the cache memory, as B is read column by column, from top to bottom. In this project, we will explore various ways to address this issue and also analyze what happens when nearly all the numbers are zero.

Research questions:

1. How much time is saved by loop reordering and block-based multiplication compared to the basic dense matrix multiplication method?

2. In what cases is multiplication using the CSR format faster and less memory-intensive than dense matrix multiplication?

Hypotheses (prior to taking measurements):

1. Both optimized methods will be faster than the original, and the performance gap will widen as n increases.

2. The CSR format will perform better when the number of elements is low and worse when it is high, as the workload increases in proportion to the square of the density; furthermore, each operation incurs a higher cost due to the indirect way in which data is accessed.

2. Context and references

Changing the loop order. If we use the i-k-j order, the innermost loop traverses a row of B and a row of C sequentially; this allows us to make better use of each cache line loaded into memory. The operations performed by the program remain the same as in the i-j-k order; the only difference is the order in which they are executed.

Block-based multiplication (or cache blocking) involves dividing matrices into smaller blocks—for example, 64 × 64—and then performing the multiplication block by block. This allows the data currently in use to fit into the cache memory, where it can be reused before being evicted.

The CSR (Compressed Sparse Row) format stores only the non-zero elements using three arrays: one for the values, one for the column index of each value, and a third for the starting position of each row. When multiplying two matrices in CSR format, the operation is performed row by row, with each resulting row being constructed in a temporary array.

If the non-zero elements are uniformly distributed with a density d, multiplication using the CSR format requires approximately $n^3 \cdot d^2$ operations, compared to the n^3 operations required for standard matrix multiplication. The expected density of the resulting matrix is $1 - (1 - d^2)^n$, which increases very rapidly as n grows.

3. Metodology

3.1 Implementations

Everything is in Java. The basic method is reused from Assignment 1 (`MatMul.java`, validated there) as `multiplyBasic` in `Dense.java`.

Method	File	What changes from the basic version
basic i-j-k	`Dense.java`	Nothing (baseline from Assignment 1)
i-k-j	`Dense.java`	Loop order: B and C are traversed row by row
Blocked	`Dense.java`	i-k-j aplicado a bloques de 64 × 64
CSR	`Sparse.java`	CSR format and CSR × CSR multiplication implemented from scratch, without libraries

All methods use 64-bit doubles and a single thread.

3.2 Input data

- **Part A:** The same matrices from Task 1, made using `generate.py` (using Python's `random` module with a seed value of 42 and numbers spread evenly across the range [0, 1)). The sizes are: 100, 200, 400, 800, and 1600.
- **Part B:** Matrices built in Java with `java.util.Random` (seed 42 for A and 43 for B). Each element shows up as non-zero based on a probability d, with numbers spread evenly in the range [0, 1); so the non-zero numbers are spread evenly. The densities used are: 0.1%, 1%, 5%, 10%, 25%, 50%, 75%, and 100% (sparsity = 100 − density). The sizes are: n = 1000 and 2000.
- Both the regular version and the CSR version do the multiplication using the exact same matrices.

3.3 Environment

Element	Value
CPU	AMD Ryzen 5 5625U, 6 núcleos / 12 hilos, 2.3 GHz

Element	Value
	base, 16 MB L3
RAM	16 GB DDR4-3200
OS	Windows 11 Home 10.0.26200
Java	OpenJDK 21.0.12.1 (Temurin), HotSpot Server VM, `-Xmx4g`
Python (scripts)	3.11.9, numpy 2.2.6, pandas 3.0.1, matplotlib 3.10.6

Todas las medidas se tomaron el 26 de octubre de 2026 en una sola tanda.

3.4 Validation

- The three dense methods were compared against NumPy (numpy.allclose, rtol = 1e-9, atol = 1e-12) for n = 2, 3, 10, and 100, as well as for the cases A × I = A and A × 0 = 0.
- The CSR format was compared with the dense version using the same matrices (n = 100) with densities of 0, 1, 10, 50, and 100%.
- All tests were successful. Furthermore, the verification value for C matches between the dense version and the CSR format across all tested configurations.

3.5 Measurement protocol

- **Stopwatch:** `System.nanoTime()` is used to measure time. The result is recorded, and the multiplication is performed. Reading or creating the input data is not included in the measurement.
- **Conversion to CSR:** This conversion is measured independently, during the same iteration, immediately before the multiplication in CSR format is performed.
- **Repetitions:** 3 warm-up multiplications are performed (which are not counted), followed by 5 measured repetitions for each configuration. The results are summarized using the median and the interquartile range (IQR).
- **Memory:** The estimated size of arrays A, B, and C is calculated. In dense format, they occupy 8 bytes per element; in CSR format, they occupy 12 bytes for each non-zero value (8 bytes for the value and 4 bytes for the column index), plus 4 bytes for each row. Memory used by the JVM is not included.
- **Time limit:** A maximum of 600 seconds is allocated per multiplication. Since all configurations finished within this limit, the largest size tested in each part (1600 and 2000) also represents the maximum valid size.

4. Results

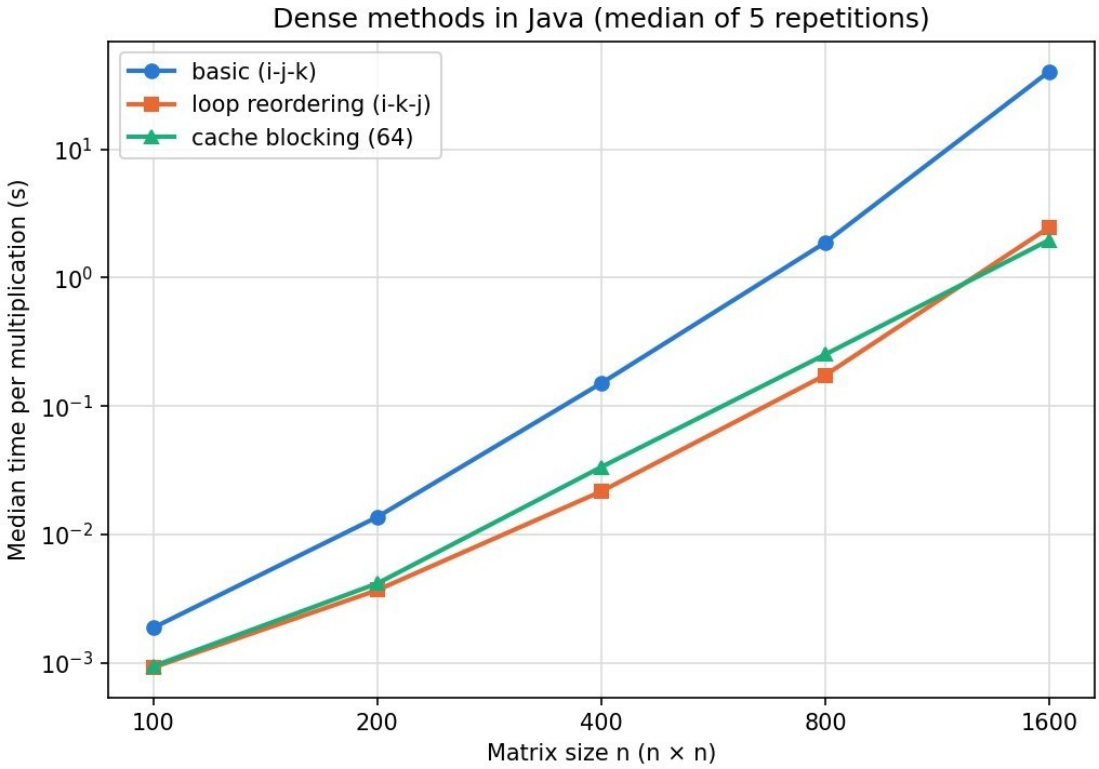
4.1 Part A: dense optimizations

Table 1. Median time per multiplication (s) and improvement relative to the basic version (in parentheses). 5 repetitions.

n	basic i-j-k	i-k-j	blocked

100	0.0019	0.00092 (2.0×)	0.00095 (2.0×)
n	**basic i-j-k**	**i-k-j**	**blocked**
200	0.0137	0.0037 (3.7×)	0.0042 (3.3×)
400	0.151	0.0217 (7.0×)	0.0336 (4.5×)
800	1.88	0.174 (10.8×)	0.253 (7.4×)
1600	40.3	2.46 (16.4×)	1.97 (20.5×)

The interquartile range (IQR) was less than 7% of the median in 9 out of the 15 configurations. The smallest sizes exhibit the greatest variability—where a single multiplication takes only a few milliseconds—and the i-k-j approach with n = 1600 (IQR of 23%) also shows significant variation. Nevertheless, for n = 1600, the 5 block-based repetitions (1.95–2.11 s) were faster than the 5 repetitions of the i-k-j approach (2.33–2.97 s). Memory usage for the arrays is identical across all three methods: $3 \times 8n^2$ bytes (58.6 MiB for n = 1600).



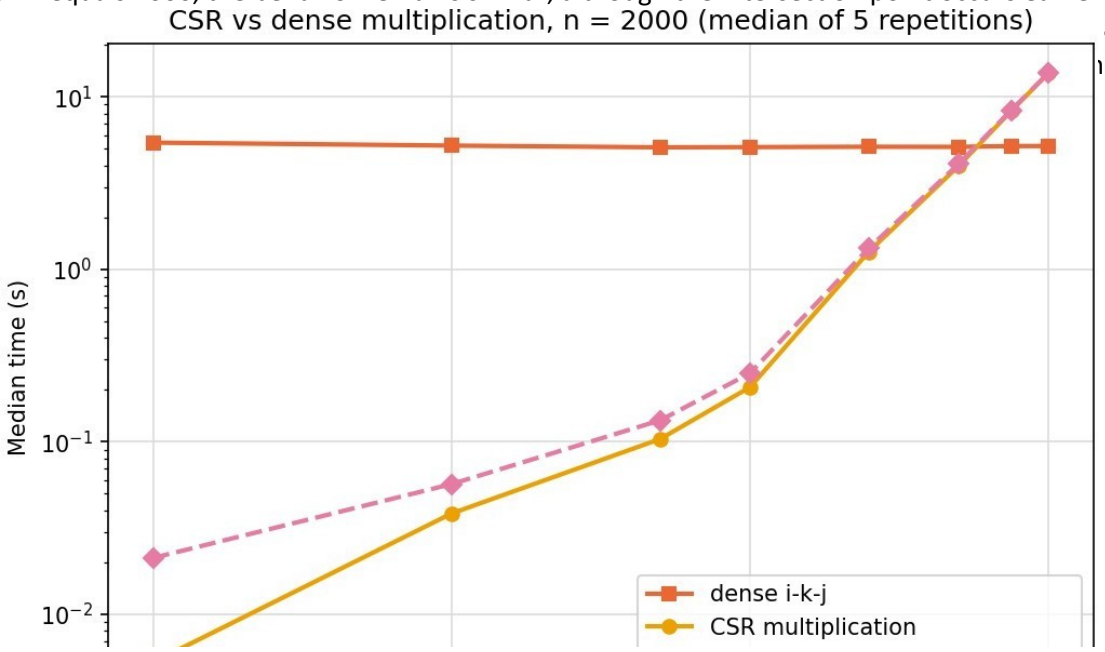
4.2 Part B: CSR vs. Dense

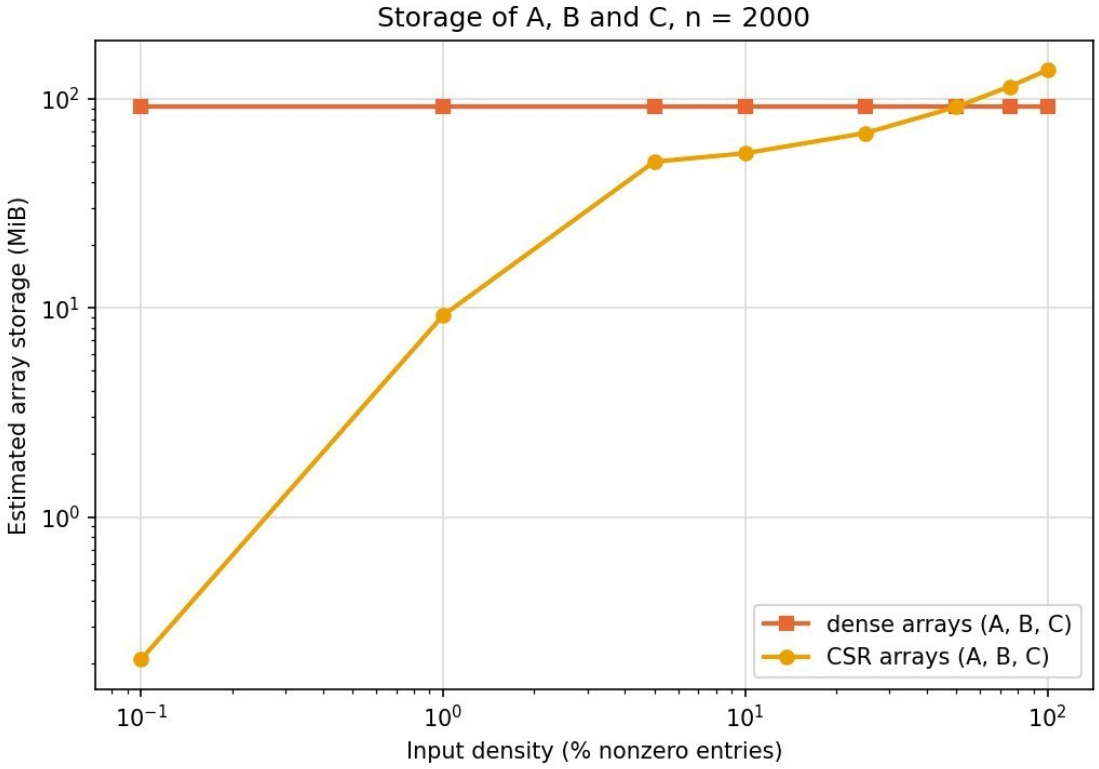
Table 2. n = 2000. Median time (s), result density, and estimated memory for A, B, and C (MiB).

Input density	Dense i-k-j	CSR multiplication	CSR conversion	Density of C	Dense memory	CRS memory
0.1 %	5.42	0.0053	0.016	0.2 %	91.6	0.2

1 %	5.21	0.038	0.018	18.1 %	91.6	9.2
5 %	5.09	0.104	0.029	99.3 %	91.6	50.0
Input density	**Dense i-k-j**	**CSR multiplication**	**CSR conversion**	**Density of C**	**Dense memory**	**CRS memory**
10 %	5.11	0.207	0.042	100 %	91.6	54.9
25 %	5.13	1.26	0.061	100 %	91.6	68.6
50 %	5.12	3.97	0.109	100 %	91.6	91.5
75 %	5.18	8.25	0.084	100 %	91.6	114.5
100 %	5.18	13.67	0.043	100 %	91.6	137.4

When n equals 1000, the behavior remains similar, although the intersection point occurs earlier: at a a dens ı find





5. Discussion

Dense optimizations. Both optimizations perform the same operations as the original method; therefore, all performance improvements stem from how memory access is managed. In the *i-k-j* approach, the innermost loop traverses the rows of B and C consecutively, rather than jumping between different rows. This improvement becomes more pronounced as *n* increases, since the original method is increasingly affected by a higher number of cache misses. The blocked version is slightly slower than the *i-k-j* approach for values of *n* up to 800—due to the overhead introduced by the additional loops—but it begins to show an advantage when *n* = 1600. At this size, matrix B (approximately 20 MB) no longer fits into the 16 MB L3 cache; consequently, the *i-k-j* approach must repeatedly retrieve matrix B from RAM for each row *i*, whereas the blocked approach reuses each segment while it remains in the cache. This explanation aligns with the size at which this effect manifests, although it has not been verified using hardware counters.

Sparse matrices. The execution time for the dense version does not vary based on density, as it always performs n^3 operations. The time for the CSR (Compressed Sparse Row) format increases approximately with d^2 , as predicted by the model; furthermore, each individual operation is more costly due to indirect access via column indices, a larger volume of data to process, and the traversal of temporary rows. Consequently, at 100% density, the CSR format is between 2.6 and 4 times slower than the dense format. If, at this density, the CSR format is c times slower, the crossover point occurs at $d \approx \sqrt{(1/c)}$ approximately 50% for n = 1000 and 62% for n = 2000—which aligns with the results obtained experimentally.

Conversion and output. Converting from a dense format to CSR takes less than 0.11 seconds in all cases. This is only significant at very low densities, where the multiplication process is so fast that the conversion itself can take longer than the calculation. The resulting matrix fills with non-zero elements much more rapidly than the input matrices; for instance, with an input density of 5% and n
= 2000, the resulting matrix C already contains 99% non-zero elements (the model $1 - (1 - d^2)^n$ predicts 99.3%). Therefore, in terms of memory usage, the CSR format ceases to be advantageous at a density near 50%; beyond this point, storing C in CSR format requires more memory than in dense format (12 bytes per element versus 8).

Limitations.

- A single piece of equipment was used: specifically, a laptop with variable processing speed. The main procedure, with n = 1600, took 40.3 s in this run and 35.8 s for Task 1; by contrast, a previous test conducted with an older version of the software took 21.0 s. Measurements are precise when compared within the same run, although absolute execution times may vary from one day to the next.
- Only one block size (64) and one type of distribution (uniform) were evaluated. If the non-zero values were clustered—as is the case with banded matrices—performance could vary significantly.
- The figure for memory usage is an approximation of the space occupied by the arrays; it does not account for the Java Virtual Machine or auxiliary arrays.
- Interpretations regarding cache memory behavior have not been verified using hardware-level measurement tools.

6. Conclusions

On this machine and with these implementations:

1. Simply changing the order of the loops (i-k-j) makes dense matrix multiplication between 2 and 16 times faster, without modifying the operations themselves. The blocked version only outperforms the i-k-j approach when matrix B no longer fits into the L3 cache (n = 1600); at that point, a 20-fold increase in speed is achieved.

2. With non-zero elements distributed uniformly, the CSR (Compressed Sparse Row) format is significantly faster than the dense format when the density is low; however, it ceases to be advantageous at densities between 25% and 50% (for n = 1000) and between 50% and 75% (for n = 2000). The conversion cost is minimal.

3. The percentage of zeros, on its own, is not a sufficient basis for making a decision: the result quickly becomes dense, which reduces the memory savings offered by the CSR format.

Further, more comprehensive research is required regarding the impact of block size, other sparsity patterns, and the parallel version; all these aspects will be addressed in Task 3.